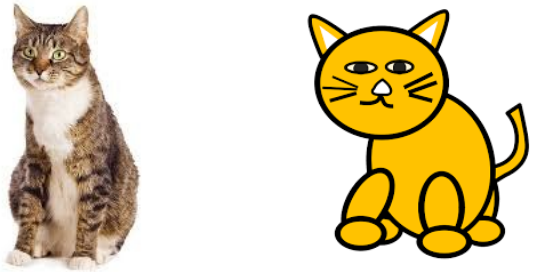
Images and Embeddings

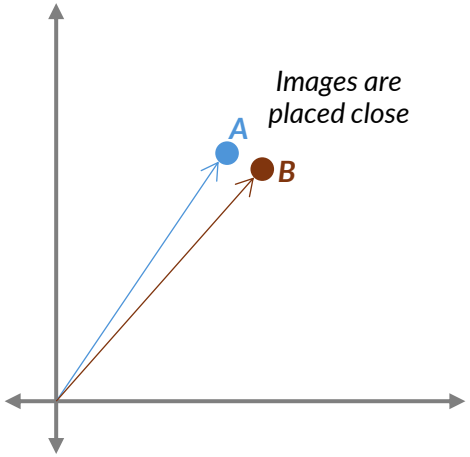
NEED FOR REPRESENTATION

- The core of computer vision is not classification or segmentation, but **representation**
- Raw images are high-dimensional, structured signals where each pixel value is locally meaningful but globally ambiguous
- Small changes in pixels can correspond to no semantic change
- Semantically similar images may differ greatly at the pixel level
- Therefore, learning directly from pixels is inefficient
- Neural networks work well mainly because they learn progressively better representations of data



A *B*

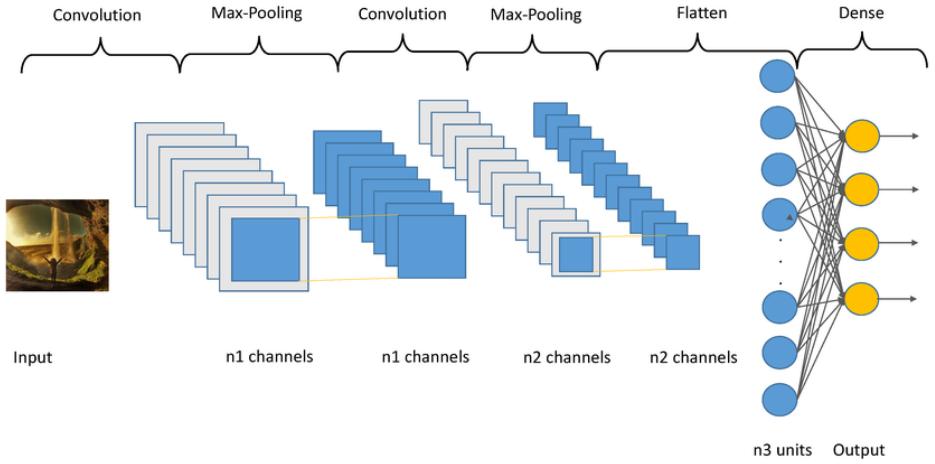
Different images pixel-wise, but both are the same entity



We need to represent these images in a way that includes meaning

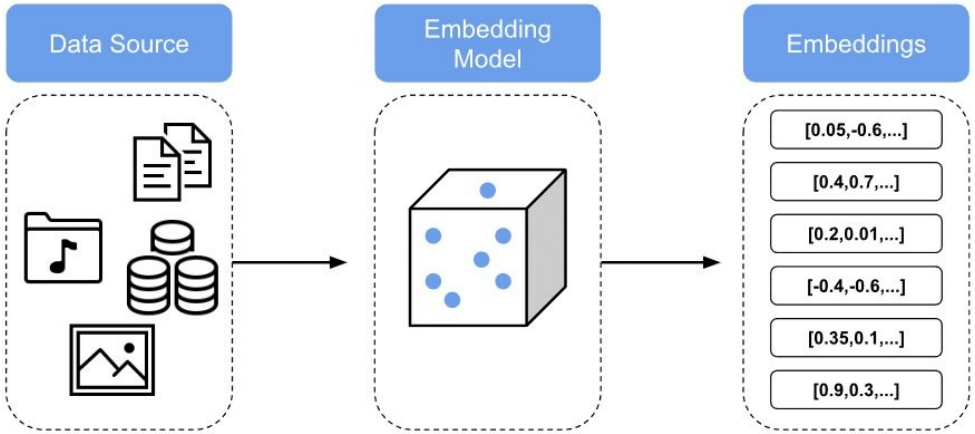
IMAGES AS NUMERICAL OBJECTS

- An image is a tensor of numbers, typically $(H \times W \times C)$ *(height, width, colour)*
- While precise for computation, these numbers do not capture the underlying “meaning”
- We cannot say pixels closer together are more “related”
- This motivates a transformation from pixel space to a representation space, like feature vectors
- These feature vectors act as compressed summaries of images
- Embeddings formalise and generalise this idea



Early CNN layers learn low-level features, deeper ones capture more abstract, and final layers produce ***fixed-length feature vectors*** *used for prediction*

WHAT IS MEANT BY EMBEDDING?



An embedding is a fixed-length vector of continuous numbers that represents the "essence" of an input

- Embedding an input means mapping it from its original space into a **continuous, lower-dimensional vector space** where structure and similarity are preserved
- Formally,

$$f: \mathcal{X} \rightarrow \mathbb{R}^d$$

  - $\mathcal{X}$ is the input space (images, text, audio)
  - $\mathbb{R}^d$ is the embedding space
  - $d \ll |\mathcal{X}|$

- Embedding is *a task-informed transformation*, not just compression, which preserves important relationships

ENCODING DATA TO EMBEDDINGS

- The encoding process transforms high-dimensional data into a lower-dimensional **latent space*** where similar concepts are mathematically close to one another
- The layer within a model that converts the raw data to vector representations are called **encoders**
  - For example, the CNN layers acts as the encoder that extracts features as vector embeddings
- In an embedding space, semantic relationships are reflected in **distance** and **direction**
- This allows model to *reason*, *rank*, and *retrieve* using relative distances
- The components of this embedded vector correspond to abstract, latent features learned by the model during training

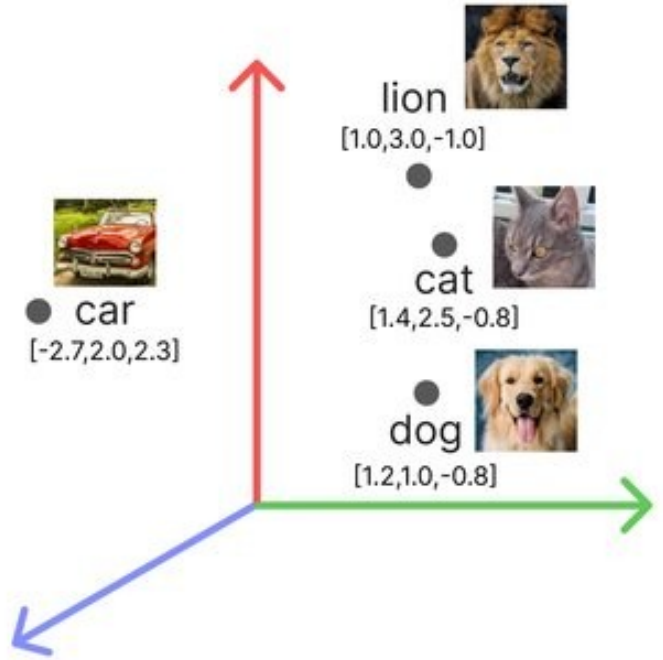


- But why don't we use the other naïve representations?
  - *One-hot encodings*: sparse, high-dimensional, no notion of similarity
  - *Pixel arrays*: dense but semantically meaningless
- These have no smooth notion of similarity and are inefficient for downstream tasks

A latent space is a **learned, unobserved representation space in which data are encoded so that underlying structure or factors of variation are captured* – embedding space is a latent space

PROPERTIES OF EMBEDDING SPACES

- An ideal embedding space exhibits:
  - **Semantic Similarity**: Distance corresponds to meaning instead of pixels
  - **Continuity**: Small changes in input lead to small changes in embedding
  - **Clustering**: Natural grouping of similar concepts without explicit labels
  - **Dimensional Efficiency**: High information density per dimension
- These properties make embeddings usable beyond the original task – classification, retrieval, clustering, search, transfer learning



EMBEDDINGS, FEATURES AND ACTIVATIONS

To understand how AI "sees," we need to distinguish between the different types of numeric data flowing through a neural network

Embeddings translates the entire unstructured input (pixels) into a single point in an abstract latent space

- Generally, they are dense vectors and much lower in dimensions
- They don't represent "where" things are, but "what" things are

Activations are the raw outputs of a neuron or a layer after an activation function (like ReLU) has been applied

- They represent the network's immediate response to an input at a specific point in the architecture
- They are often sparse and high-dimensional

Features are the specific attributes or patterns that the network has learned to recognise

- They are the concepts being responded to (by activations), defined by the weights of filters
- Early layers learn simple physical features while deeper ones learn more abstract features

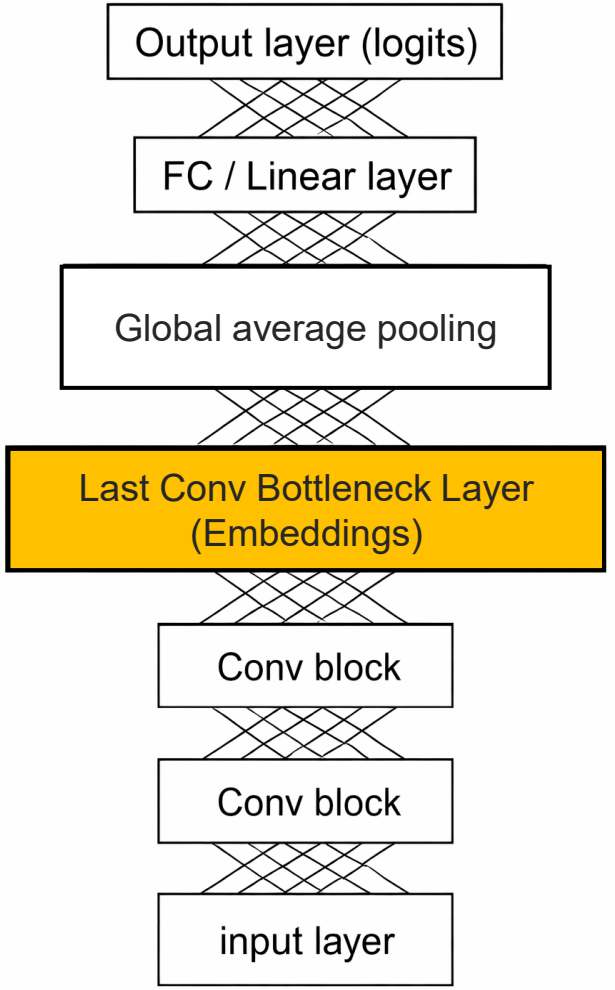
For example: A feature is the “template” for a shape. The activation is the signal indicating how strongly the image embedding aligns with that feature.

Embeddings don’t represent “where”. But what if output needs structure?

Encoder Decoder Architectures

WHEN SINGLE VECTORS ARE NOT ENOUGH

- Embeddings compress images to vectors
- This works well when the output itself is simple, such as a class label, a score, or a fixed-length decision
- But some tasks require structured outputs, not just a single vector or label
- In these cases, we need predictions that:
  - Preserve spatial structure
  - Depend on local and global context
  - Produce outputs of the same or similar resolution as the input
- Here, collapsing the spatial embeddings into a single vector becomes a bottleneck



Here the spatial feature maps turns into a vector

Provides rich, compact representation before task-specific collapse in pooling and FC layers

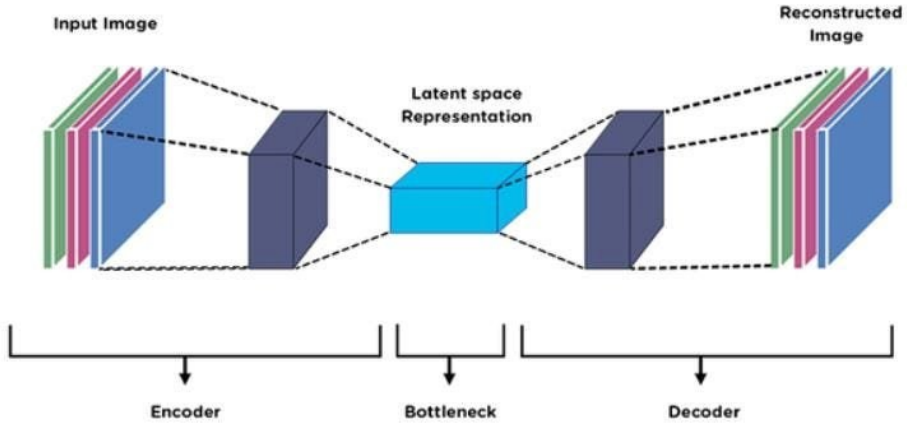
PRESERVING SPATIAL INFORMATION

- Spatial information encodes meaning in structured vision tasks like:
  - Segmentation (labelling pixels)
  - Depth estimation
  - Image translation and super-resolution
- A single embedding vector loses fine-grained spatial information and cannot directly express pixel-level correspondences
- In a CNN, we have: ***Image → Features → Pooling → Vector → Prediction***
- Pooling and flattening improve invariance and reduce computation, but it also discards spatial details
- This collapse is effective for classification, but unsuitable for dense prediction (prediction at each spatial location) tasks
- We want to separate understanding (semantic meaning from the input) and generating structured outputs aligned with the input
- This leads to the encoder-decoder paradigm, which helps to avoid or delay the collapse

ENCODER-DECODER ARCHITECTURE

An encoder–decoder model consists of two complementary components:

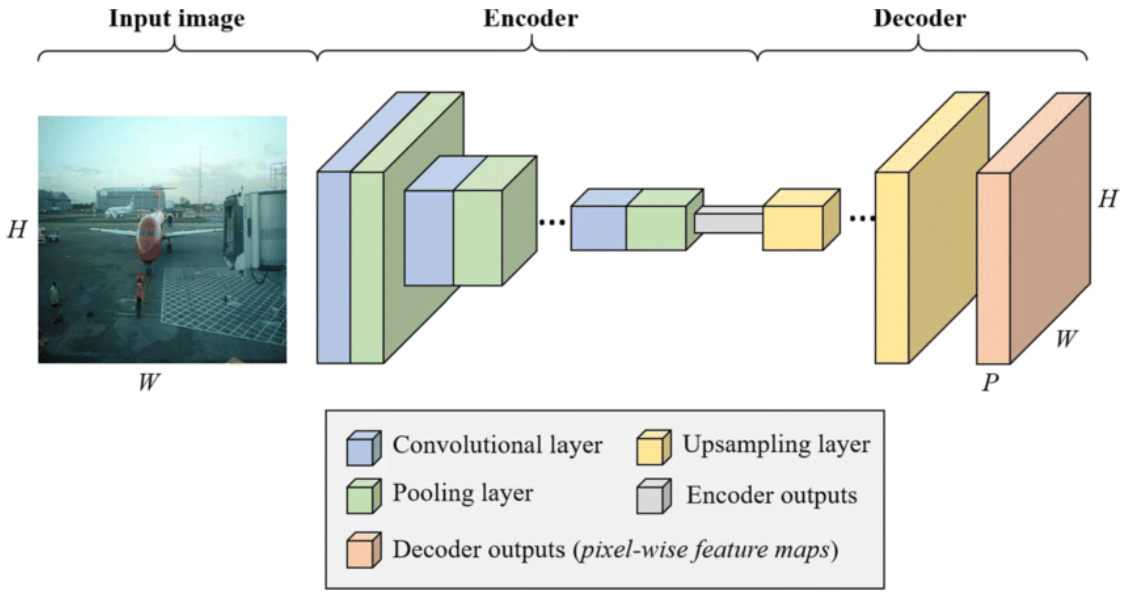
- **Encoder**
  - Transforms input into a compressed latent representation
  - Learns what is present
  - Focuses on abstraction and semantics
- **Decoder**
  - Transforms latent representation into a structured output
  - Learns where things belong
  - Focuses on spatial reconstruction
- Together, they form a representation–generation pipeline



ENCODER-DECODER WORKING

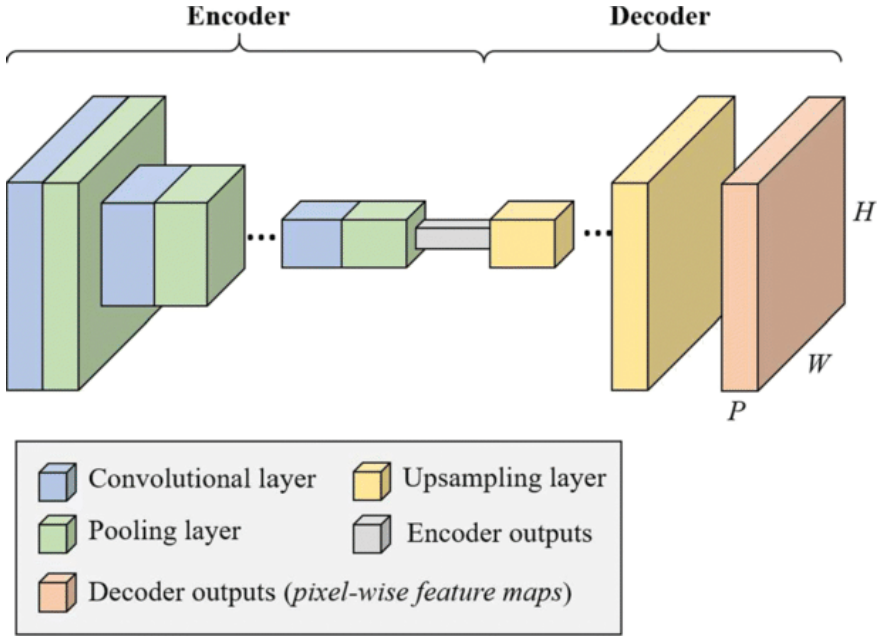
1. The encoder progressively reduces spatial resolution while increasing semantic richness
2. The latent representation captures global context
3. The decoder progressively restores resolution while mapping semantics back to spatial structure

- This works better than predicting directly from raw features, as encoder-decoder models
  - Separate abstraction from localisation
  - Scale better to complex outputs



ENCODER

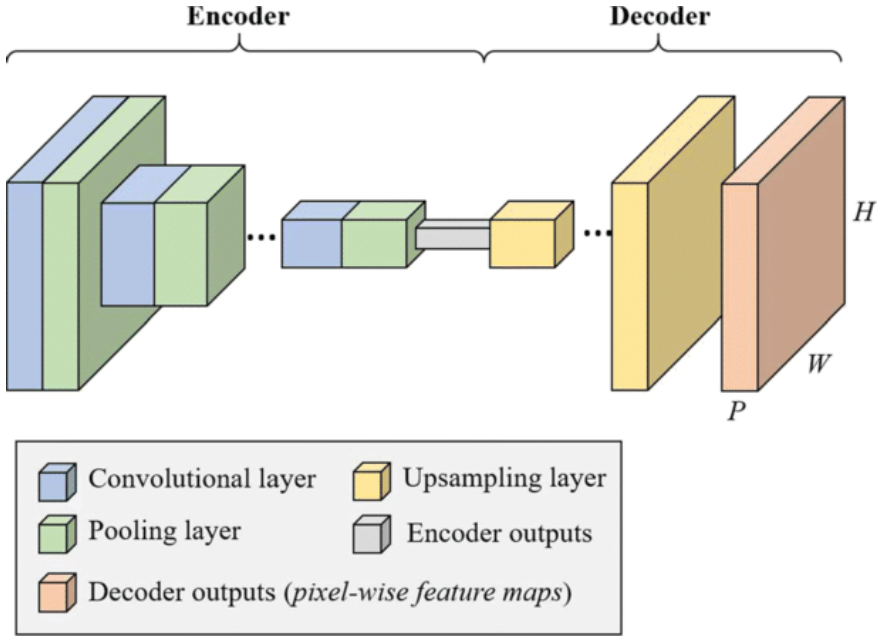
- Encoder is a stack of learnable layers that transforms raw input into compact latent representation
- Reduces the size (height, width) while simultaneously increasing the complexity and depth of the feature maps
- Early layers capture local patterns; deeper layers capture global, abstract features
- Final output is low-resolution, high-channel, information-dense
- Acts as the information bottleneck and decides what information to retain



- An encoder typically consists of convolutional layers, non-linear activations, normalisation and pooling layers
- It produces a spatial embedding (feature map), not a single vector
- Feature maps can be converted into embeddings via operations that remove spatial structure (pooling)

DECODER

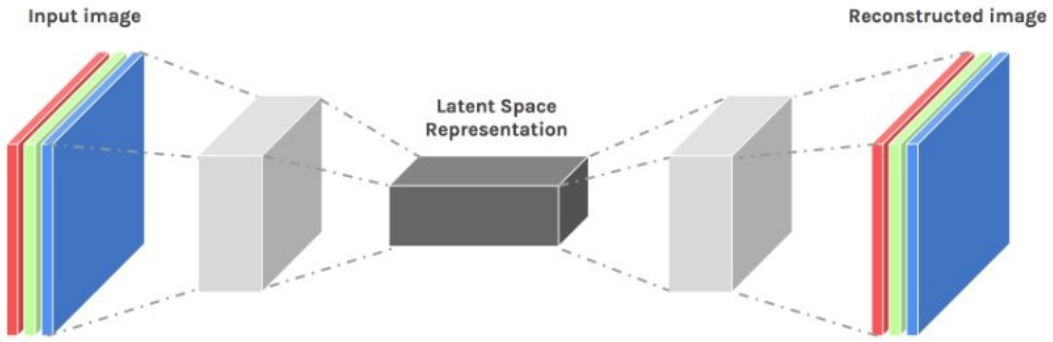
- Decoder, also a stack of learnable layers, transforms latent representations into structured outputs
- Increases spatial resolution (height, width) while refining and reducing feature abstraction
- Combines global semantic information with local spatial detail
- Progressively reconstructs task-relevant structure from compressed representations
- Maps latent features back to the output space while preserving alignment with the input



- A decoder typically consists of upsampling layers, convolutional layers, non-linear activations and normalisation
- It operates on spatial embeddings and produces structured predictions, not single values
- Spatial detail lost during encoding cannot be fully recovered through decoding alone

BOTTLENECK (LATENT SPACE VECTOR)

- The point of maximum compression in the middle of the network is known as the Bottleneck Layer or the Latent Space Vector (or Context Vector)
- It is the most compressed, low-dimensional numerical representation of the input image
- It forces the encoder to extract only the most essential, abstract, and semantically meaningful features
- By forcing this compression, the network learns to ignore noise and redundant information.
- The decoder must rely solely on this abstract representation to generate the final output
- It ensures a true understanding of the input rather than just memorisation



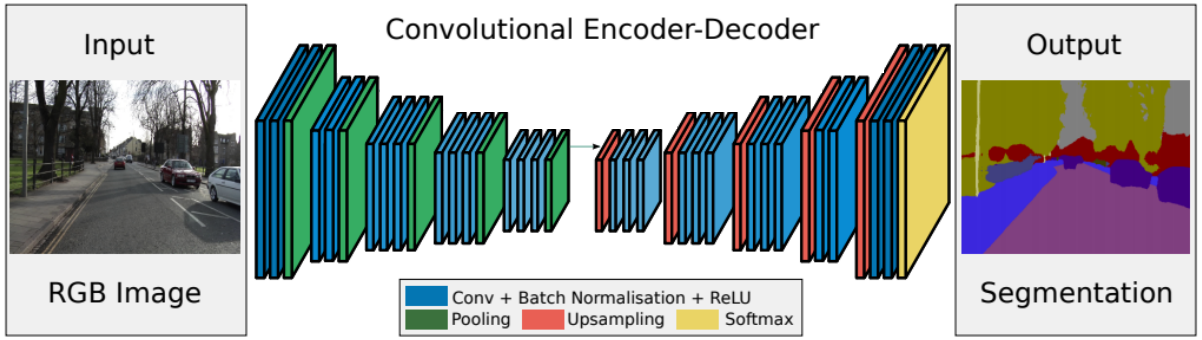
USE CASES

- The encoder-decoder architecture is effective when
  - input and output have similar structure but different meaning
  - output resolution is important
  - global context can influence local predictions
- They are less useful for scalar/categorical outputs and when spatial structure is not as important
- Uses across domains:
  - Computer vision: segmentation, depth, restoration, denoising, image generation
  - NLP: machine translation, summarisation, text generation
- It provides interpretable intermediate representations and is extensible across domains
- However, it comes with a higher computational cost
- A careful architecture design is needed as decoder quality depends on encoder maps, which have a risk of losing fine details in deeper architectures
- These shortcomings motivated further improvements

U-Net for Segmentation

ENCODER-DECODER SEGMENTATION

- Encoder–decoder architectures enable structured outputs
- But deep encoders lose fine details, and decoders alone struggle to reconstruct sharp boundaries
  - Pooling and striding improve abstraction but also blurs edges, shapes and small objects
- This becomes problematic for pixel-level (dense) tasks like segmentation
- One solution for this is **U-Net**



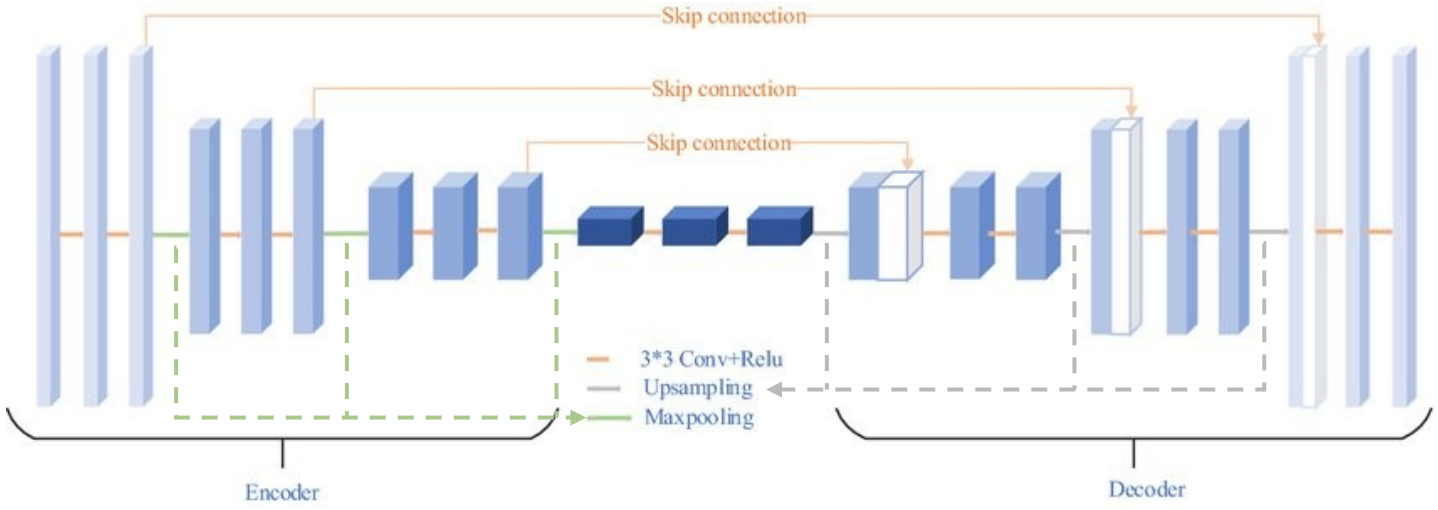
SKIP CONNECTIONS

Recall skip connections in ResNet:

- Skip connections help gradient flow
- Add identity mappings to mitigate degradation and stabilise training

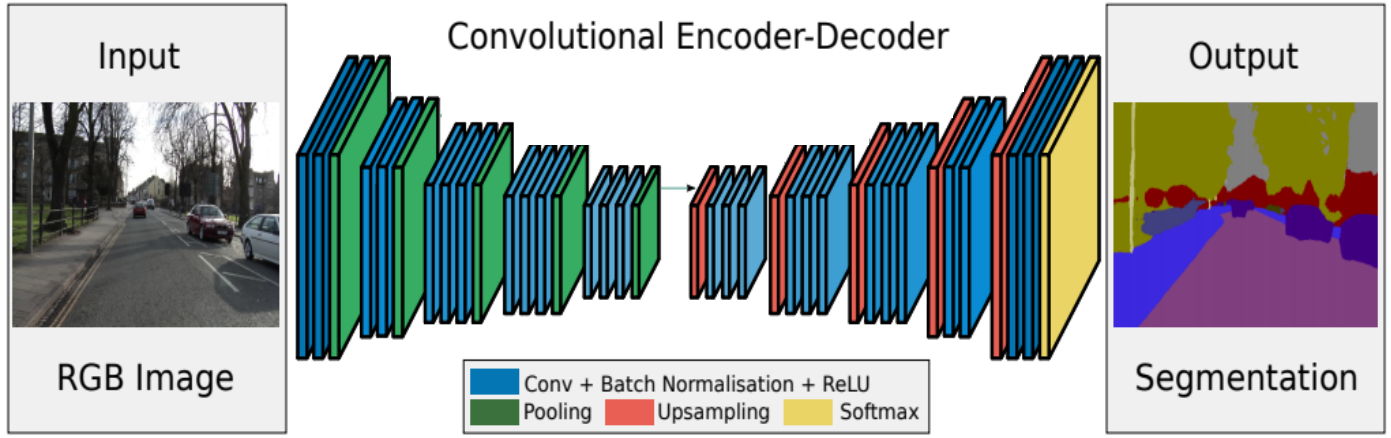
U-Net also uses similar direct connections to pass information and gradients, but with a different purpose

- Skip connections preserve spatial information
- Features are concatenated to restore detail during reconstruction
- Used for reconstruction, not optimisation



INSIGHT BEHIND U-NET

The primary goal of U-net is to perform dense prediction, where the output is an image-like mask that assigns a class or identity to every single pixel in the input image (segmentation)



Encoder: *resolution* ↓, *semantics* ↑

Decoder: *resolution* ↑, *semantics refined*

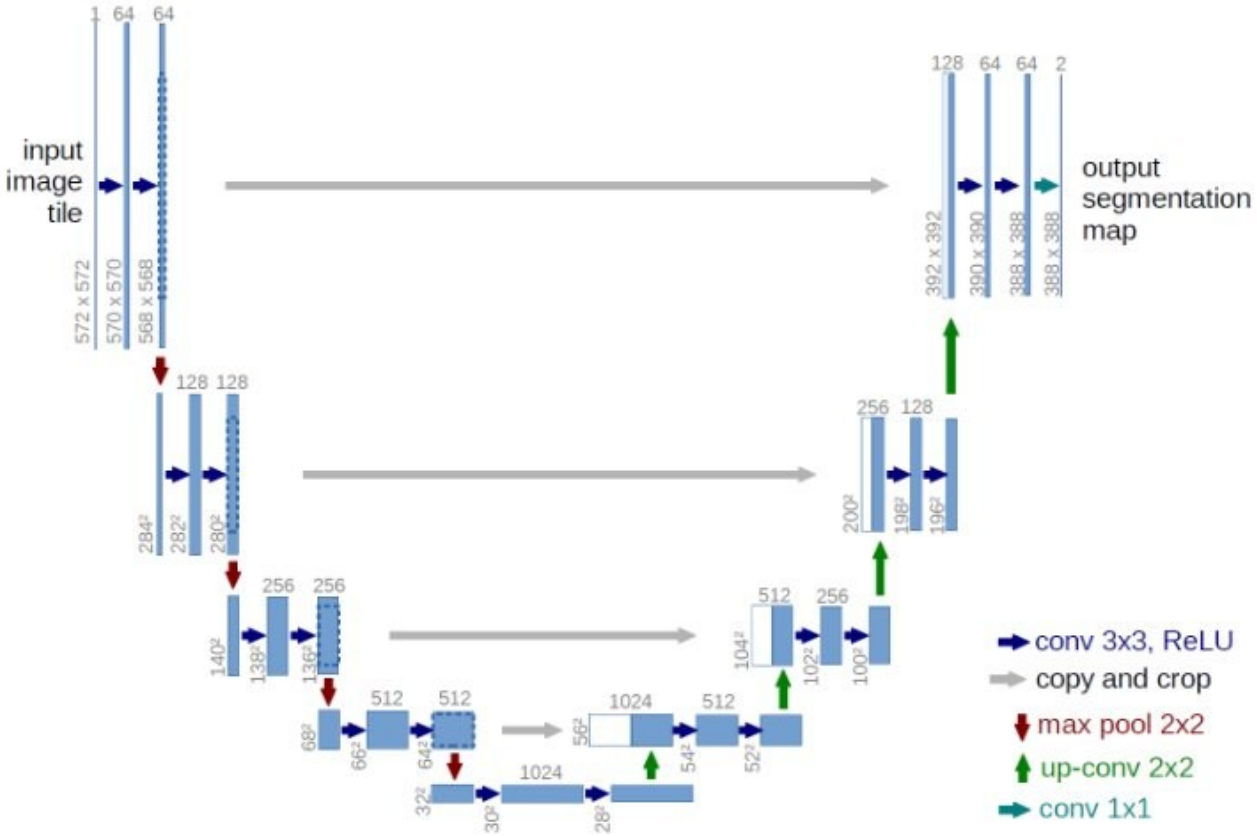
- Decoder cannot reconstruct the image accurately if the finer details are lost
- Skip connections help replenish this lost detail by providing high-resolution early stage features directly to decoder

Pooling loses spatial information that the decoder requires later. So why do we use pooling?

- Makes representations less sensitive to small translations and distortions – two cats shown from different angles should be seen as cats
- Increases effective receptive field and deeper neurons to understand larger portions of image
- Reduces feature map size and creates bottleneck to ensure only essential information passes

U-NET ARCHITECTURE

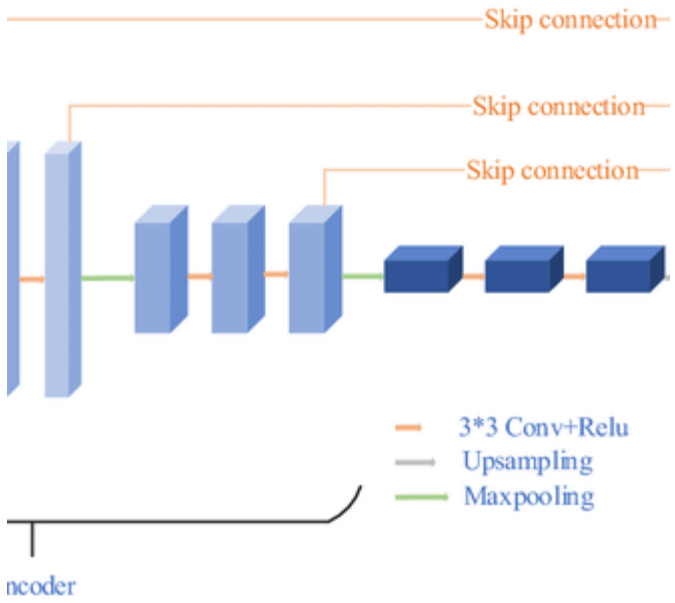
- U-Net is a symmetric encoder-decoder with skip connections
- The encoder part is called the **contracting path**
- Similarly, decoder is called the **expanding path**
- Encoder downsamples and learns what is present, while decoder upsamples and predicts where it belongs
- Skip connections link matching resolutions



"U"-Net

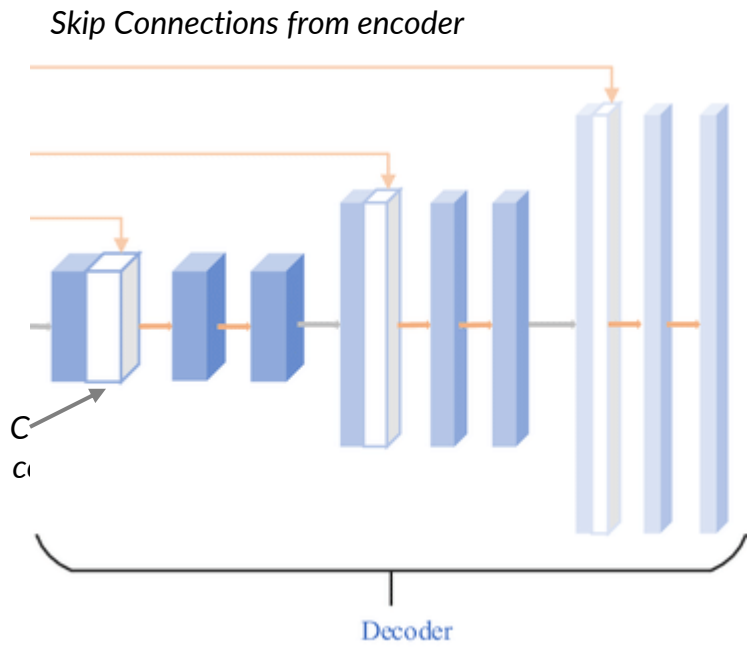
THE CONTRACTING PATH

- The contracting path (or encoder) is responsible for capturing the context and semantics of the image
- It consists of repeated blocks, typically involving:
  - 3x3 convolutions (followed by ReLU activation)
  - A 2x2 Max Pooling operation with stride 2 for down sampling (halving the spatial dimensions and doubling the number of feature channels)
- As the path contracts, the network learns increasingly abstract, location-invariant features (the "what" is in the image)



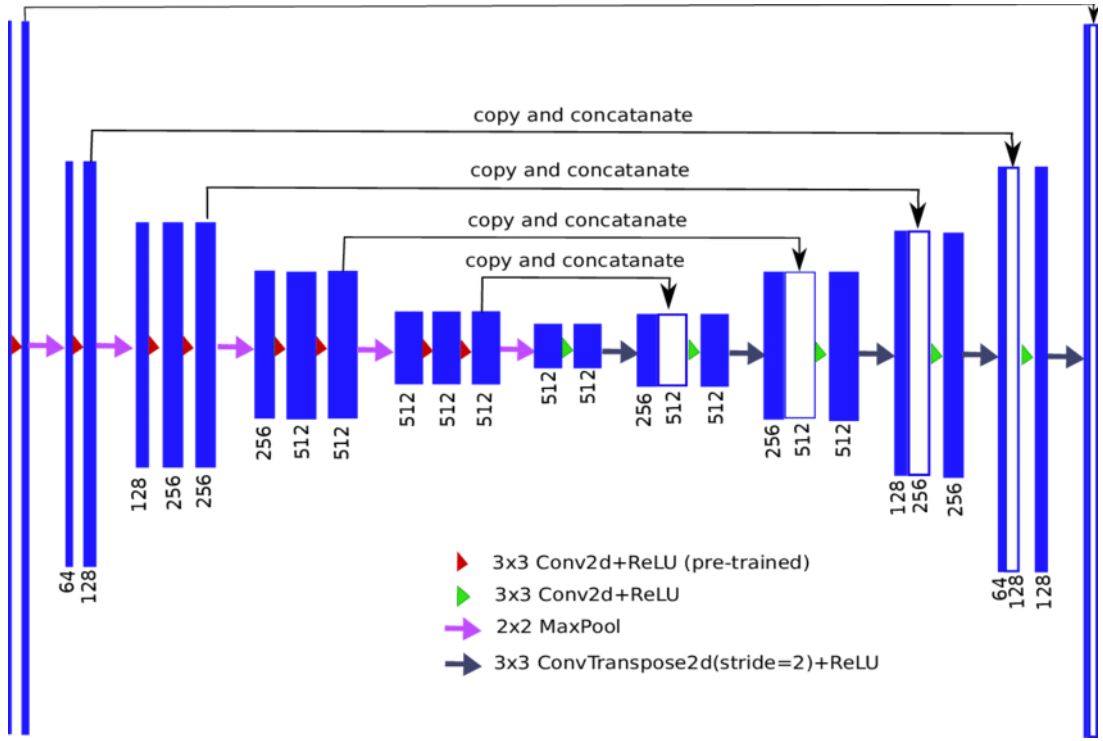
THE EXPANDING PATH

- The decoder expanding path is responsible for precise localisation and reconstruction of the segmented mask
- It mirrors the contracting path
- Each step typically involves:
  - An upsampling of the feature map, which doubles the spatial dimensions and halves the number of feature channels
  - Concatenation with the feature map from the corresponding skip connection
  - Two 3x3 convolutions + ReLU
- As the path expands, the network combines semantic information with high-resolution detail to produce the final mask (the "where" exactly are the objects)



WORKING OF SKIP CONNECTIONS

- Direct connections that bypass the bottleneck and transfer feature maps from the encoder to the decoder at the same resolution level
- These are important as encoder loses spatial detail
- Decoder uses global context from deep layers and local detail from shallow layers
- This concatenation preserves information richness, boundary details and ensures decoder output is highly localised



*The feature map from the encoder is cropped and **concatenated** depth-wise with the upsampled feature map from the decoder*

WHY U-NET

- Segmentation requires predicting what is present and where exactly, at the pixel level
- Pooling and deep encoding capture semantic meaning but lose precise spatial detail

Advantages

- U-Net preserves object boundaries and fine structures using skip connections
- High-resolution encoder features enable accurate pixel alignment with global context
- Strong architectural bias reduces data requirements, effective in data-limited domains
- Best suited for dense, spatially aligned outputs: medical images, satellite imagery, document layouts

- However, skip connections increase memory usage and limit large or multi-scale context modelling